

ARYAN SHARMA

Network & Protocol Engineer

Ghaziabad, India · +91 93154 65182 · aryansharma24112003@gmail.com

github.com/Aryan-Protein-Vala · linkedin.com/in/aryannnn · aryannnn-portfolio.vercel.app

SUMMARY

Network and protocol engineer with end-to-end experience from silicon to application: custom binary frame protocols over raw TCP with 8 MB socket buffers, QUIC transport for WAN, mDNS zeroconf discovery, BLE GATT protocol design with MTU negotiation and fragmentation, Delay-Tolerant Networking with spray-and-wait routing, and WebSocket fabrics with sub-millisecond fan-out.

TECHNICAL SKILLS

- Protocols: TCP/IP (TCP_NODELAY, socket buffer tuning, Nagle) · custom binary framing ([lengthPrefix][binaryFrame]) · QUIC · WebSockets (RFC 6455) · BLE GATT (MTU negotiation, CCCD, onCharacteristicWrite) · mDNS/ZeroConf (ZeroConf_tcp services) · DTN & opportunistic routing (spray-and-wait)
- Crypto in transit: E2EE (AES-256-GCM, ChaCha20-Poly1305) · Noise Protocol Framework · P-256 ECDH + HKDF · ECDSA signatures · replay-attack defense (Bloom-filter dedup)
- Architecture: P2P + relay/signaling hybrids · LAN auto-bridging (mDNS intercept) · mesh networking · relay/hub patterns · Pub/Sub fan-out · 24/7 service operations (health endpoints, keep-alive, cold-start avoidance)
- Languages: Go · Rust · Swift · Kotlin · Python · C (background) · Linux networking

EXPERIENCE

Cinntra Infotech — Systems Engineer · 2026 – Present

- Built CIRA, a RAG agent translating natural-language requests into audited SAP HANA queries and write operations, giving non-technical users direct access to enterprise data
- Built the point-of-sale system backend: transaction recording, merchant reconciliation, and offline synchronization

Niswa Engineering — Technical Consultant & Automation Engineer · 2024 – 2026

- Built asynchronous AI agents (TypeScript) that automated 80%+ of a content generation and scheduling pipeline
- Built the company website in Astro (95+ Google Lighthouse score); set up cloud infrastructure, API integrations, and event-driven automation workflows

Sudarshana Foundation — Full-Stack & Automation Engineer · Contract

- Designed and built the organization's first centralized web platform, digitizing paper-based workflows into software
- Built database systems handling 5,000+ monthly transactions

PROJECTS

Aether — Custom P2P Transfer Protocol (Go)

- Designed a protocol stack that saturates physical links: raw TCP multiplexer with persistent TCP_NODELAY sockets, 8 MB OS socket buffers, and a [lengthPrefix][binaryFrame] protocol carrying 16 MB parallel chunks — modeled 110–115 MB/s on Gigabit, 700–1,100 MB/s on 10G (71 MB/s measured peak).
- Built the discovery + handoff layer: an invisible _aether._tcp mDNS ZeroConf signature broadcasts locally; on detecting a same-room peer, the engine intercepts the relayed connection and establishes a direct LAN bridge — transfers then run above ISP ceilings without public IP routing.
- Tech: Go, raw TCP, QUIC, mDNS/ZeroConf, AES-256-GCM — github.com/Aryan-Protein-Vala/AetherNet

Astral — BLE Mesh Protocol + DTN Routing (Rust/iOS/Android)

- Designed a BLE payment protocol that defeats standard BLE payload limits: dynamic MTU negotiation up to 512 B, then a sequential queuing system over onCharacteristicWrite for zero-packet-loss fragmentation of 200–500 B encrypted payloads — including a manual CCCD (0x2902) write fix that resolved the Android "merchant did not respond" bug breaking iOS peripheral ACKs.
- Built the Delay-Tolerant core: every phone becomes a roaming router using spray-and-wait opportunistic routing — messages hop over Bluetooth as devices pass, E2E-encrypted (P-256 ECDH + ChaCha20-Poly1305) so intermediate carriers can't read them, and reconcile to a ledger when either endpoint reconnects.
- Tech: Rust, Swift, Kotlin, UniFFI, BLE GATT, Noise Protocol, DTN — github.com/Aryan-Protein-Vala/Astral-Offline · github.com/Aryan-Protein-Vala/Astral-Network

CLAIM/05 — WebSocket Fabric for 25M-Cell Real-Time State

- Built the WebSocket fabric in Go (Gorilla, hub pattern, concurrent channel hubs): persistent bidirectional sessions, delta-only updates over Redis Pub/Sub (grid_updates channel) fanned out to all connected clients in sub-millisecond, SETNX atomic locking for conflict-free writes, and reload-safe token reconnection.
- Tech: Go, Gorilla WebSockets, Redis — github.com/Aryan-Protein-Vala/Grid-Game · live: multiplayer-grid-game-theta.vercel.app

V■YU — WebSocket Latency Protocol Tuning

- Profiled a production WebSocket path and found the protocol-level bug: missing TCP_NODELAY on uvicorn's websockets path let Nagle + delayed ACK add ~42 ms per round trip — fixed at the socket layer and verified 44 ms → 1.5 ms (96% reduction) with a scripted 10-query audit harness.
- Tech: Python, FastAPI, Uvicorn — github.com/Aryan-Protein-Vala/vayu-voice-rag

CERTIFICATIONS

- Frontend Development Certification — Sheryians Coding School
- Data Science & AI/ML Bootcamp Certification — Udemy